Computing Fundamental
Experiment # 11

While Do-While Loop

1. Objective/s:

1. While Loops
2. Do-While Loops
3. Lab Tasks

2. Loops

Loops are very useful when you want to perform a task repeatedly. Loop's body has set of statements, which is executed on every iteration. We have three types of loop control structures in C, even though working of these following loops are almost similar still there is a major difference among them. You may need to choose one based on the requirement.

1. **for loop:** This is most commonly used loop in C language. The syntax and flow of this loop is simple and easy to learn. However, there are few cases when you may prefer any other loop, instead of this.
2. **while loop**: This is used when you need to repeatedly execute the block of statements, for fixed number of times. Read this tutorial to understand the flow of this loop.
3. **do-While loop:** It is similar to the while loop, the only difference is that it evaluates the test condition after execution of the statements enclosed in the loop body.

2.1. While Loops

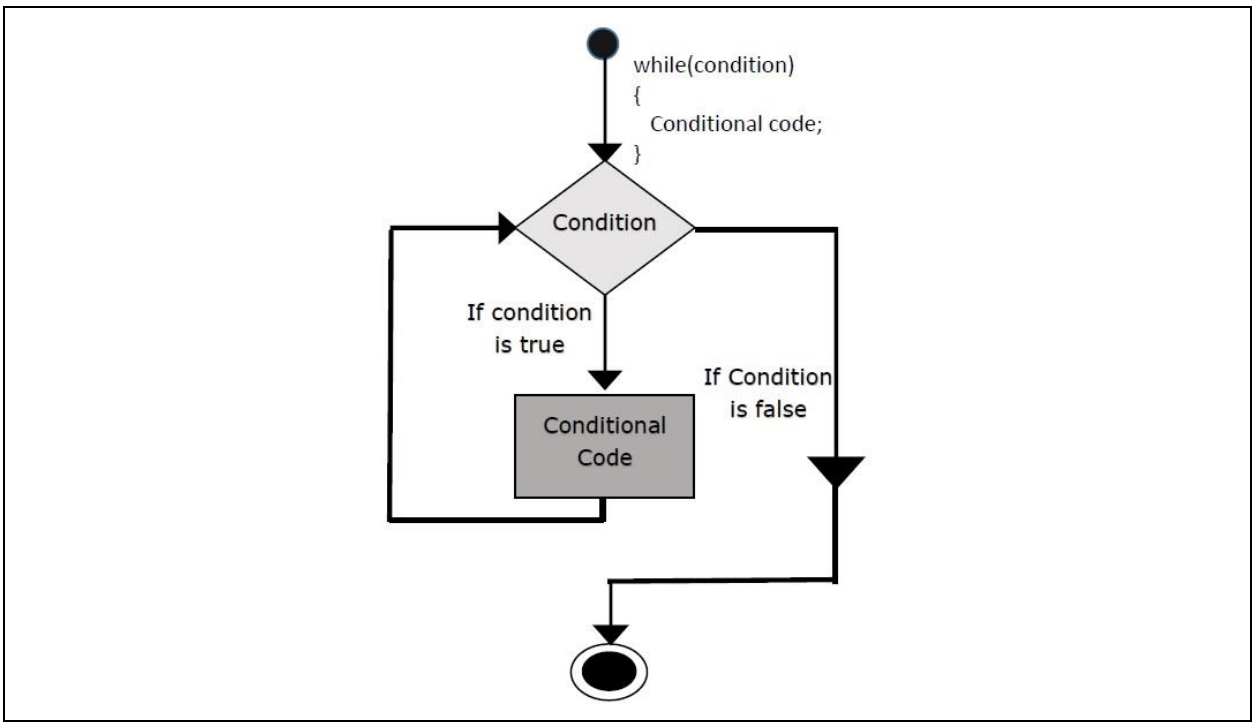
2.1.1. Syntax

```
while (testExpression)
{    //codes
}
```

The while loop evaluates the test expression.

If the test expression is true (nonzero), codes inside the body of while loop is executed. The test expression is evaluated again. The process goes on until the test expression is false. When the test expression is false, the while loop is terminated.

2.1.2. Flow Chart



2.1.3. Example

```
// Program to find factorial of a number
// For a positive integer n, factorial = 1*2*3...n

#include <stdio.h>
int main()
{
    int number;
    long long factorial;

    printf("Enter an integer: ");
    scanf("%d",&number);

    factorial = 1;
    // loop terminates when number is less than or equal to 0
    while (number > 0)
    {
```

```
        factorial *= number;  // factorial = factorial*number;
        --number;
    }
    printf("Factorial= %lld", factorial);

    return 0;
}
```

2.2. Do…While Loops

The do..while loop is similar to the while loop with one important difference. The body of do..while loop is executed once, before checking the test expression. Hence, the do..while is executed at least once.

2.2.1. syntax

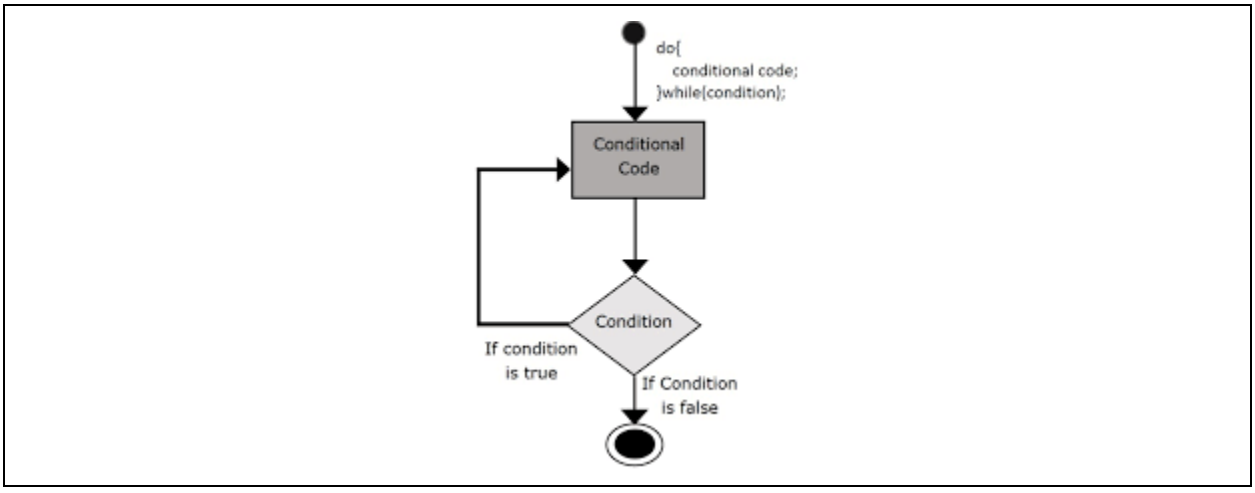
```
do
{
   // codes
}
while (testExpression);
```

The code block (loop body) inside the braces is executed once.

Then, the test expression is evaluated. If the test expression is true, the loop body is executed again. This process goes on until the test expression is evaluated to zero (false).

When the test expression is false (nonzero), the do...while loop is terminated.

2.2.2. Flow chart



2.2.3. Example

```c
// Program to add numbers until user enters zero

#include <stdio.h>
int main()
{
    int number, sum = 0;

    // loop body is executed at least once
    do
    {
        printf("Enter a number: ");
        scanf("%d", &number);
        sum =sum+ number;
    }
    while(number != 0);

    printf("Sum = %d",sum);

    return 0;
}
```

3. Lab Tasks

1. Construct program that displays the inverse pyramid with * using while loop.

```
*
**
***
****
*****
```

2. Write a C++ program to find a reverse of a number entered by user for example reverse of 231 is 321.

3. Write a program for Counting Digits and displaying their total count at the end of the program using while loop.

4. Write a program that asks user for a number and checks if the number is Armstrong Number or not. (e.g. 153 is an Armstrong Number)